

Grammatical Error Correction

Objective

Model Grammatical Error Correction as Text-to-Text Translation problem, translating from noisy, error-prone text into fluent, grammatically correct text. Design and build an end-to-end Grammatical Error Correction application using an encoder-decoder transformer. Rather than predicting token-by-token edits, you will finetune the model to translate "noisy" or ungrammatical text directly into fluent English. The final deliverable will be a working prototype (e.g., a web interface) and an evaluation of its performance.

1. Core Tasks

Phase 1: Data Preparation and preprocessing

[3 marks]

- Use agentlans/grammar-correction dataset.
- Perform standard NLP preprocessing as applicable, so that your model can be fine tuned.
- Apply appropriate padding / task pre-fixing as needed by your model.
- apply appropriate tokenization.

Phase 2: Model Fine-Tuning & Application Build

[5 Marks]

- **The Model:** Fine-tune a lightweight pre-trained model like **t5-small** or **t5-base** using the Hugging Face transformers library.
- use appropriate learning rate and epochs.
- You are free to use any other transformed model suitable to achieve this goal.
- Implement early stopping to halt training if the validation loss stops improving for 2 consecutive epochs
- **The Application:** Wrap your fine-tuned model in a simple web UI using **Gradio** or **Streamlit**. Your application should take a user's text input, run it through the model, and display the corrected sentence on the screen.

Phase 3: Handling Over-Correction

[1 Mark]

Because sequence-to-sequence models are built to generate entirely new text (like in translation or summarization), they often suffer from **over-correction** in Grammatical Error Correction tasks. A model might take a slightly awkward but grammatically correct sentence and completely rewrite it just to make it sound "better," destroying the user's original voice.

You must **discuss and demonstrate** how you mitigated over-correction in your application.

Identify 1–2 test samples where the model over-corrected.

Phase 3: Evaluation

[4 Marks]

- **BERTScore:** Use BERTScore to measure the semantic similarity between your model's output and the ground-truth corrected sentence. This ensures the model didn't hallucinate or completely change the user's intended meaning.
- **Character/Word Error Rate (CER/WER):** Calculate the edit distance between the source sentence and the generated sentence.
- Provide 5 clear examples of the model succeeding, and 3 examples of the model failing (e.g., hallucinating a word or missing an obvious error), with a brief explanation of why those failures might have occurred.

2. Deliverables:

- A fully executable Jupyter notebook with clear mention of all the members of your team including Name and Bits ID.
- There should not be any error cell, and all the cells should be executable.
- Screenshots with test cases showing the inferences and predictions of the given words.
- Screenshots showing use of BITS CSIS Labs.
- Any queries related to this assignment should be addressed to Course LF, Ajay Naik